

...FIRST: What Problem Does Bazel Solve? (Very Important)



Real-World Analogy: Factory Assembly Line

Imagine you run a **factory** that builds **cars** 🚗

❌ Old Style (Make / CMake)

- Workers pick **random tools**
- Parts are taken from **anywhere**
- Sometimes a car builds, sometimes it breaks
- One worker changing a bolt breaks the whole factory
- Rebuilding **everything** every time



This is how **traditional build systems behave**



✅ Bazel Style

- Every worker:
 - Knows **exact inputs**
 - Produces **exact outputs**
 - Works in **isolation**
- If one bolt changes:
 - Only **affected parts** rebuild
- Same inputs → **same output every time**



This is **Bazel**



CORE IDEA (Burn This Into Your Brain)

Bazel builds a dependency graph and only rebuilds what actually changed.

Everything else is just implementation details.



STEP 1 — Install Bazel (Quick)

On Ubuntu:

```
sudo apt install bazel
```

Verify:

```
bazel --version
```

STEP 2 — Your First Bazel Project (HELLO WORLD)

Directory Structure

```
hello_bazel/  
├── WORKSPACE  
├── BUILD  
└── main.cpp
```

WORKSPACE (Project Root Marker)

```
# WORKSPACE
```

Analogy

WORKSPACE = **Factory boundary**

“Everything inside belongs to this project”

Even an **empty file is valid**.

main.cpp

```
#include <iostream>

int main() {
    std::cout << "Hello Bazel 🚀" << std::endl;
    return 0;
}
```

BUILD (This is the HEART of Bazel)

```
cc_binary(
    name = "hello",
    srcs = ["main.cpp"],
)
```

BUILD File Explained (Line by Line)

1 cc_binary

“I want to build a **C++ executable**”

Other examples:

- cc_library → C++ library
 - cc_test → C++ test
-

2 name = "hello"

Target name

Like naming a **product** in factory

You'll refer to it as:

```
//:hello
```

3 srcs = ["main.cpp"]

Exact input files

Bazel **forces you to declare inputs**

! If you forget a file → **build fails**

→ This is GOOD (correctness)

Build It

```
bazel build //:hello
```

Output:

```
bazel-bin/hello
```

Run it:

```
./bazel-bin/hello
```

WHAT JUST HAPPENED INTERNALLY?

Bazel did this:

1. Read `BUILD`
 2. Created a **node** called `//:hello`
 3. Saw inputs: `main.cpp`
 4. Compiled in **isolated sandbox**
 5. Cached result
-

STEP 3 — Incremental Build (Bazel Magic)

Run build again:

```
bazel build //:hello
```

⚡ **Instant** — nothing rebuilt

Now edit `main.cpp` :

```
std::cout << "Hello Bazel World 🌍" << std::endl;
```

Build again:

```
bazel build //:hello
```

➡ Only **this target rebuilt**

🧠 Why This Is Powerful

- Bazel hashes inputs
 - If unchanged → **reuse output**
 - No “clean build” nonsense
-

🟢 STEP 4 — Add a Library (Real Project Structure)

📁 New Structure

```
hello_bazel/  
├── WORKSPACE  
├── BUILD  
├── main.cpp  
└── math/  
    ├── BUILD  
    ├── math.cpp  
    └── math.h
```

📄 math/math.h

```
#pragma once  
int add(int a, int b);
```

📄 math/math.cpp

```
#include "math.h"
```

```
int add(int a, int b) {  
    return a + b;  
}
```

math/BUILD

```
cc_library(  
    name = "math",  
    srcs = ["math.cpp"],  
    hdrs = ["math.h"],  
    visibility = ["//visibility:public"],  
)
```

New Concepts Explained

cc_library

“Build a reusable C++ library”

hdrs

Header files exposed to users

visibility

Who is allowed to use this library

Update root BUILD

```
cc_binary(  
    name = "hello",  
    srcs = ["main.cpp"],  
    deps = ["//math:math"],  
)
```

Update main.cpp

```
#include <iostream>
#include "math/math.h"

int main() {
    std::cout << "2 + 3 = " << add(2, 3) << std::endl;
    return 0;
}
```

Build Again

```
bazel build //:hello
```

VERY IMPORTANT RULE (Bazel Discipline)

 This is **NOT** allowed:

```
#include "../math/math.h"
```

 Only this:

```
#include "math/math.h"
```

Why?

Bazel enforces **clean dependency boundaries**

This **prevents spaghetti builds**.

STEP 5 — Labels (How Bazel Thinks)

Label	Meaning
//:hello	hello target in root
//math:math	math library

Label	Meaning
:math	same package

Think:

Labels = GPS coordinates for code

Mental Model (Lock This)

- BUILD files describe **WHAT**
- Bazel figures out **HOW**
- You declare **inputs**
- Bazel guarantees **correct outputs**

WHAT YOU NOW KNOW (FOR REAL)

- ✓ What Bazel is
- ✓ WORKSPACE vs BUILD
- ✓ cc_binary / cc_library
- ✓ deps & visibility
- ✓ Incremental builds
- ✓ Why Bazel is strict

NEXT LESSON (Tell Me Yes)

If you say “**continue**”, next we’ll cover:

Lesson 2

- Why Bazel **forbids undeclared headers**
- Header include paths
- `includes` vs `hdrs`
- Common beginner mistakes
- Mapping Bazel ↔ CMake (VERY useful)

Just reply:

continue lesson 2